ITU-MiniTwit

DevOps, Software Evolution and Software Maintenance
BSc Group F (gruppe 1)

Sebastian Holt Svendsen <sesv@itu.dk>
Andreas Nygaard Schiøtt <anys@itu.dk> Member Three <xxxx@itu.dk>
Member Four <xxxx@itu.dk> Member Five <xxxx@itu.dk>

May 16, 2026

Word count:
2801 (errors:2)

Contents

1 System's Perspective **2**
1.1 Architecture and Design . . . 2
1.2 Dependencies . . . 5
1.3 Current State of the Systems . . . 6

2 Process' Perspective **8**
2.1 CI/CD Pipeline . . . 8
2.2 Monitoring . . . 10
2.3 Logging . . . 11
2.4 Security Hardening . . . 12
2.5 Availability and Scaling . . . 13

3 Reflection Perspective **14**
3.1 Evolution and Refactoring . . . 14
3.2 Maintenance . . . 14
3.3 DevOps Style of Work . . . 15

4 Use of Generative AI **16**
4.1 Tools Used . . . 16
4.2 Impact on Work Process . . . 16
4.2.1 Supported . . . 16
4.2.2 Hindered . . . 16
ITU-MiniTwit GitHub Repository

1 System's Perspective

Authors: Sebastian Holt Svendsen

In this section, we provide an overview of the ITU-MiniTwit system from a technical perspective. We cover three areas: the architecture and design (section 1.1), the dependencies and technologies used (section 1.2), and the current state of the system (section 1.3). The app is deployed on DigitalOcean as a Docker Swarm cluster, fronted by two nginx load balancers, with a shared Redis for user-session-state and a managed PostgreSQL database for persistent data.

1.1 Architecture and Design

Figure 1 shows the production topology of ITU-MiniTwit deployed on DigitalOcean.

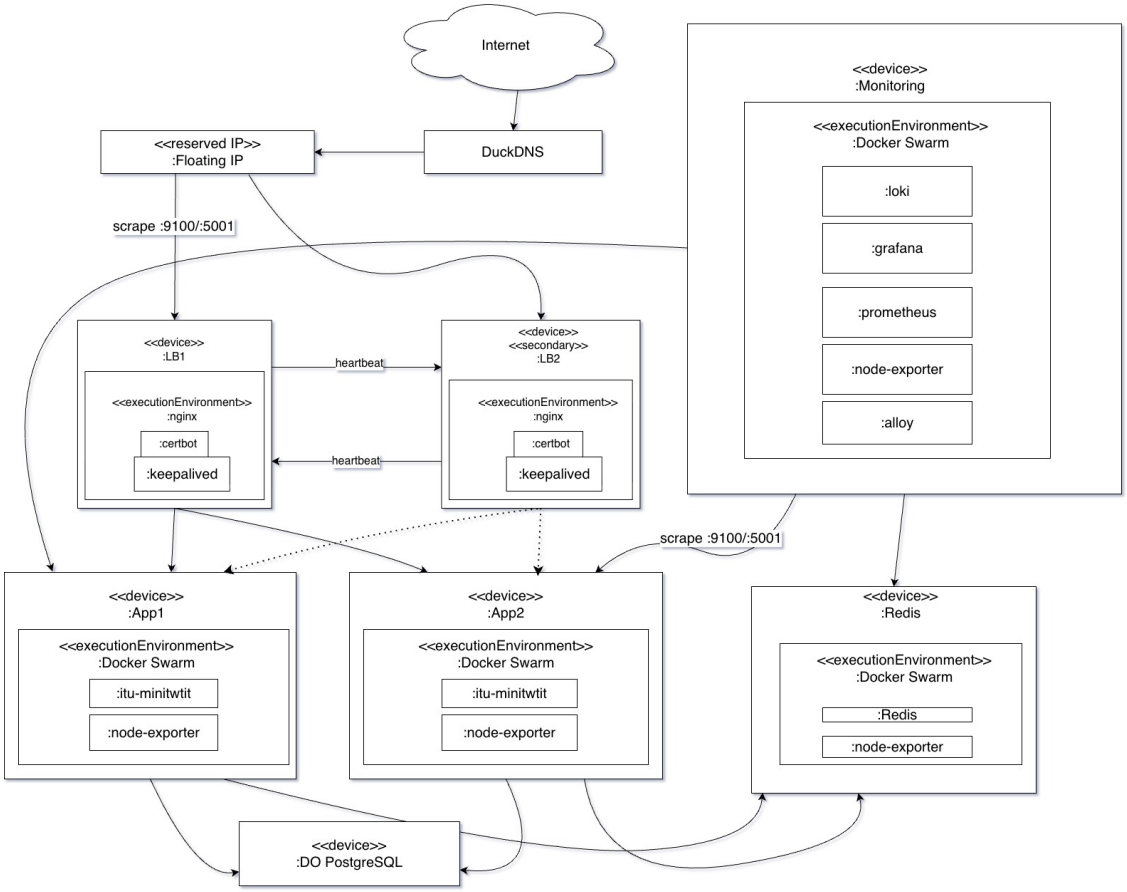

Figure 1: Deployment diagram — production topology of ITU-MiniTwit deployed on DigitalOcean.

We will be going through each tier of the diagram from top-to-bottom.

Tier 1: Public entry. Traffic enters from the internet via the DuckDNS domain. DuckDNS is a free dynamic DNS service for setting up a domain for ITU-MiniTwit. The domain points to a floating IP address, which is a public IP that can be moved between different servers in the cluster, specifically the two load balancers.

Tier 2: Load balancers are two nginx instances, LB1 and LB2, that distribute incoming requests to the webapp instances in the next tier. Two are deployed for higher availability. *Certbot* is used to automatically obtain and renew TLS certificates for the domain, ensuring

secure HTTPS connections. *Keepalived* implements the heartbeat mechanism: LB1 runs as MASTER and LB2 as BACKUP under VRRP. If LB1 stops responding, keepalived on LB2 promotes itself and the Floating IP is reattached automatically, so the public hostname keeps resolving to a healthy node without manual intervention.

Tier 3: App and data contains two app instances, the Redis instance, and the PostgreSQL database. The app instances, App1 and App2, are each a device running a Docker Swarm execution environment that hosts one `itu-minitwit` replica and a `node-exporter`. Docker Swarm schedules the replicas onto nodes labelled `role == app`. Both replicas share two backing stores: a managed PostgreSQL database for persistent data, and a Redis instance on its own node holding session state and ASP.NET DataProtection keys. Because no per-request state is kept inside the app containers, any replica can serve any user, so the load balancers can distribute requests freely and fail a node out without dropping sessions. Rolling updates similarly bring up a new container before stopping the old one, so no one is logged out. Crashed containers are restarted automatically by Swarm's `on-failure` restart policy.

Tier 4: Monitoring is its own device with its own Docker Swarm execution environment running Prometheus, Grafana, Loki, Alloy, and node-exporter. Prometheus scrapes metrics from every node — `:9100` for node-exporter (host metrics) and `:5001` for the app's `/metrics` endpoint (HTTP and runtime metrics). Grafana Alloy runs on every node, reads the local docker socket, and ships container logs to Loki, which stores and indexes them. Grafana queries Prometheus and Loki to render the monitoring and logging dashboards. Keeping the observability stack on its own node means an outage of the app tier does not also bring down monitoring, which would be needed to diagnose it.

Application architecture. Beyond the deployment topology, the application itself follows a layered design split into three .NET projects with a one-way dependency rule. `Chirp.Core` holds the domain entities (`Author`, `Cheep`, `Follows`, `Recheep`). `Chirp.Infrastructure` owns persistence, the EF Core `DbContext`, repositories, and services behind interfaces (`ICheepRepository`, `ICheepService`, etc.), and the database migrations. `Chirp.Web` is the presentation layer: ASP.NET Core Razor Pages for the user interface, scaffolded ASP.NET Identity pages for authentication, and a `MiniTwit-Stub` controller exposing the REST API consumed by the course simulator.

To complement this static view, fig. 2 traces what happens at runtime when a user requests the public timeline.

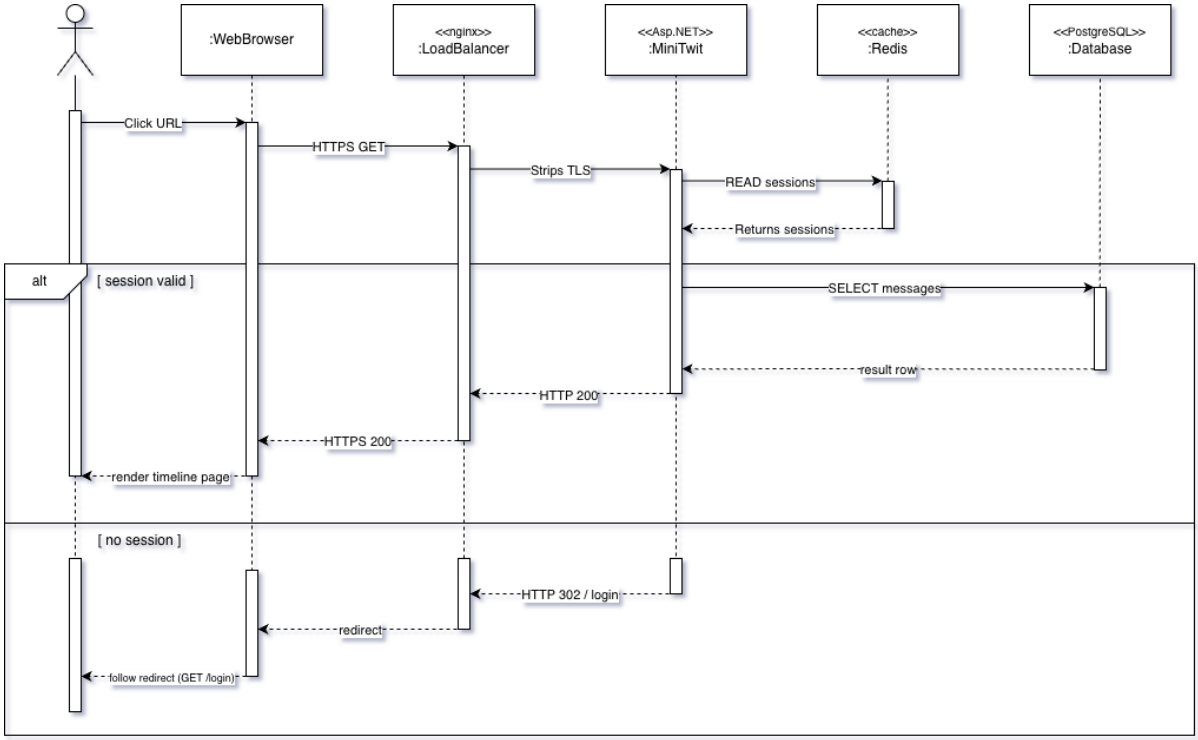

Figure 2: Sequence diagram — request flow to the public timeline.

In the common case the browser issues an `HTTPS GET`, which the active nginx instance terminates and forwards over HTTP to one of the app replicas. The replica reads the caller's session from Redis. With a valid session it queries PostgreSQL for the latest cheeps and returns the rendered timeline page with `HTTP 200` back through nginx to the browser. The diagram makes each tier's responsibility explicit: nginx only terminates TLS, the app holds no per-request state, and Redis and PostgreSQL hold all shared state.
The `alt` fragment shows the alternative flow: when no valid session exists, the replica responds with `HTTP 302 /login`. The browser follows this redirect transparently and re-enters the same flow against `/login`, so the user simply lands on the login page.

1.2 Dependencies

Tables 1 and 2 list the main technologies used in ITU-MiniTwit, grouped by the layer where they apply, from writing code through building, testing, deploying, and observing the system.

Table 1: Application dependencies, by layer.

Layer	**Technology**	**Purpose**
Language / runtime	C# / .NET 8 (ASP.NET Core)	Application language and web runtime
Web framework	ASP.NET Core Razor Pages	Server-rendered HTML UI
	ASP.NET Core Identity	Local accounts and authentication
	OAuth provider packages	Referenced but not enabled in the current build
	Swashbuckle / Swagger	Auto-generated docs for the simulator REST API
	prometheus-net.AspNetCore	Exposes the `/metrics` endpoint scraped by Prometheus
	Newtonsoft.Json	JSON serialisation for the simulator API
Data	Entity Framework Core 8 (Npgsql)	ORM; maps C# entities to tables and manages schema migrations
	PostgreSQL (DigitalOcean managed)	Persistent store for cheeps, authors and follow relations
	Redis (StackExchange.Redis)	Distributed session store and shared ASP.NET DataProtection key ring

Table 2: Infrastructure, build and operations dependencies, by layer.

Layer	Technology	Purpose
Containerisation	Docker	Containerisation of all components
	Docker Swarm	Clusters multiple Docker hosts into one
	Docker Hub	Registry for storing and distributing container images
Networking	nginx	Terminates TLS and load balances to app replicas
	certbot	Automatically obtains and renews TLS certificates for the domain
	keepalived	Implements VRRP for high availability of the load balancers
	DuckDNS	Dynamic DNS service for the public hostname
Cloud infrastructure	DigitalOcean	Droplets for all cluster nodes, the managed PostgreSQL database, and the Floating IP
Provisioning / deployment	Shell scripts	Bootstrapping, configuration, and deployment scripts for the cluster
	rsync over SSH	Copies the stack YAML files and monitoring configs to the swarm manager before deployment
CI/CD	GitHub Actions	Builds, tests and deploys the app on every push to main
Testing	xUnit	Unit and integration test framework for .NET
	Playwright	End-to-end testing of the web UI
Static analysis	SonarCloud	Code qualiy, coverage, and security analysis in CI
	hadolint + ShellCheck	Linting of Dockerfiles and shell scripts
Monitoring / logging	Prometheus	Metrics collection and storage
	Grafana	Metrics and log visualisation
	Loki	Log aggregation and indexing
	Grafana Alloy	Collects container logs and ships them to Loki
	node-exporter	Exposes host-level metrics for Prometheus
Dev tooling	Git / GitHub	Version control, code hosting, and pull-requests
	Rider	C# IDE for development
	Visual Studio Code	Latex editor used for the report

1.3 Current State of the Systems

Static analysis. Code quality is tracked continuously through SonarCloud, which analyses every push to main. The current results are shown in fig. 9. Maintainability is rated **A** (54 minor issues) and duplication is low at **4.7%**, but reliability and security both sit at **C** (20 reliability issues, 5 security issues, and 22 security hotspots awaiting review). The **test coverage is only 13.1%**, the codebase's clearest quality weakness. Dockerfiles, shell scripts and C# formatting are additionally linted in CI with hadolint, ShellCheck and `dotnet format`, configured to report findings without blocking the pipeline.

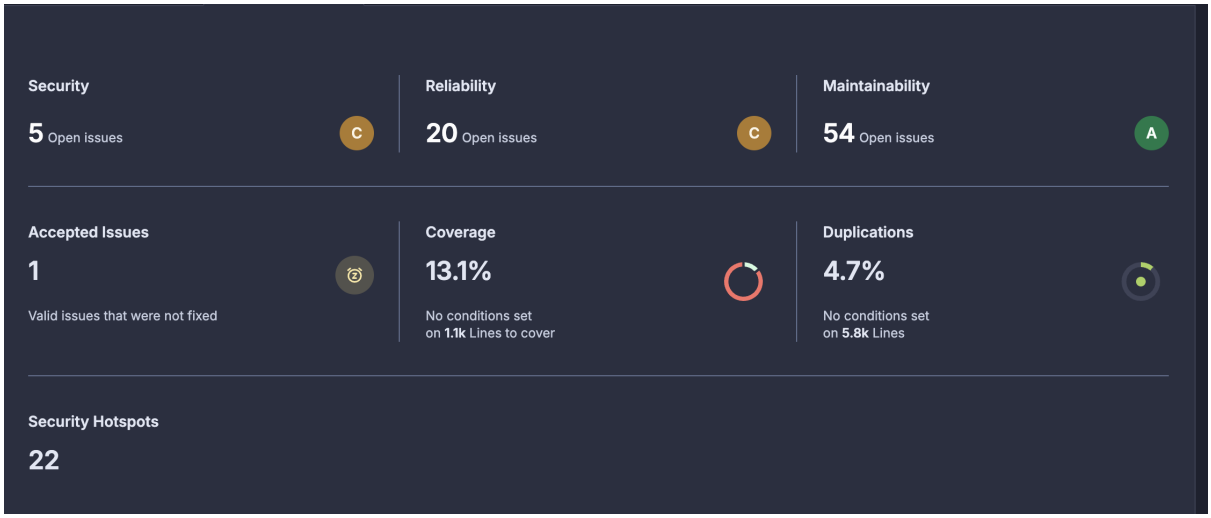

Figure 3: SonarCloud analysis summary for ITU-MiniTwit.

What works. In production the system is stable: the application reports healthy, serves the simulator's load of around **2.2k req/s** with no in-flight backlog, uses about **156 MB** of memory, and returns its API endpoints in roughly **7 ms at the median and 37 ms at the 99th percentile** with no server (5xx) errors. High availability has been demonstrated end-to-end: keepalived failover between the two load balancers, two app replicas, automatic restart of crashed containers, and rolling updates without downtime and without logging users out. The full CI/CD pipeline (build, unit and integration tests, end-to-end tests, static analysis, and deployment) is in place, and the metrics and logging stacks are deployed. Over the term the project also migrated from SQLite to a managed PostgreSQL database and from per-node Docker Compose to Docker Swarm.

Open items. A number of limitations remain, documented here:

- The public-timeline page averages about **580 ms**, far slower than the API endpoints. It's the main performance weakness, rooted in the database access pattern and round-trips to the managed database.
- The logging dashboard is dominated by Loki's own internal push errors, so application logs are not effectively surfaced, and there is no alerting.
- SonarCloud's security findings are unaddressed. The Docker images are not security-hardened and CI has no dedicated security scanner (e.g. CodeQL or Trivy).
- There is no declarative infrastructure-as-code or configuration management (Vagrant, Ansible). Provisioning is handled by shell scripts, and the `Vagrantfile` is a non-functional stub.
- With only two of five group members actively contributing, some weekly tasks (mapping the Three Ways, security scanning, infrastructure-as-code) were not reached.

These would be the priorities for further work.

2 Process' Perspective

Authors: Andreas Schiøtt

2.1 CI/CD Pipeline

Table 3: CI/CD tools

Technology	Purpose
GitHub Actions	CI/CD automation
Playwright	end-to-end testing
SonarCloud	Static analysis and code coverage
Hadolint	Dockerfile linter
ShellCheck	Shell script linter
Docker Build	Build and push container images to Docker Hub
Docker Swarm	Automated docker container orchestration
rsync	Idempotent Sync files to the remote swarm manager
Grafana	Logging / monitoring running Prometheus, loki, Alloy
Prometheus	Metric collection and monitoring
Loki	Log aggregation system
Grafana Alloy	Log and monitoring collection

Before we had a running cloud hosted droplet we only ran CI to run a docker container and run unit test on a locally hosted sqlite based application.

After cloud hosting on **DigitalOcean** was setup we added the CD pipeline to the CI pipeline. The CI/CD pipeline were both triggered concurrently on push to main branch which then builds a **Docker image**, pushes it to **Docker Hub** and then deploys it to the **DigitalOcean** droplet using **SSH & docker pull**. With the later focus on idempotent deployment, we added a check to see if the CI pipeline succeeded before the CD pipeline is triggered.

The CI pipeline was extended to include end-to-end testing using **Playwright**, and static code analysis using **SonarCloud** & **Trivy**, **Hadolint** and **ShellCheck**. As shown in fig. 4, on every push the pipeline detects which paths changed, builds the project, and then runs static anal¡¡ tests, SonarCloud analysis, and end-to-end tests in parallel.

While writing the report we itterativly extended the CI pipeline to check which files were changed to decicde which steps to run, if only the report directory was changed we only run "build PDF" report" step and vice versa.

Our pipeline also creates a scheduled weekly github release every week as a self-contained release.

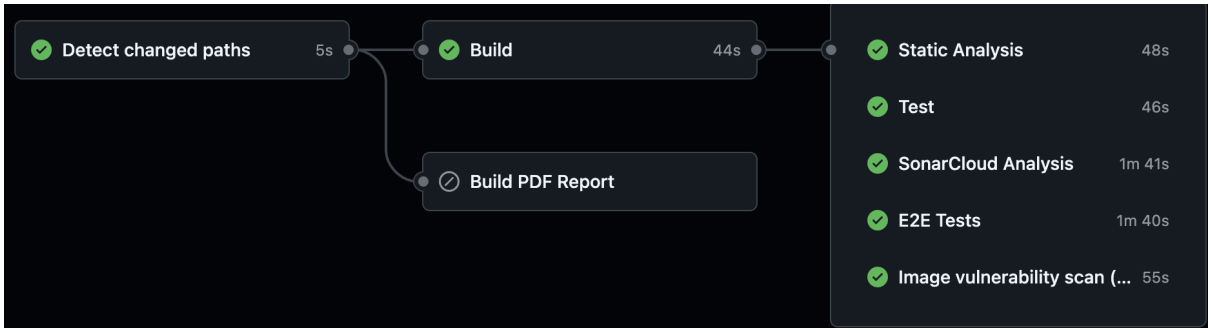

Figure 4: CI pipeline (`CI.yml`).

The CD pipeline is triggered only after a successful CI run, ensuring that broken builds are never deployed. As shown in fig. 5, it consists of three sequential jobs: building and pushing the Docker image (*build-push*), deploying the application (*Deploy*), and deploying the monitoring stack (*Deploy-Monitoring*) with **Grafana**, **Prometheus**, **Loki** and **Grafana Alloy**.

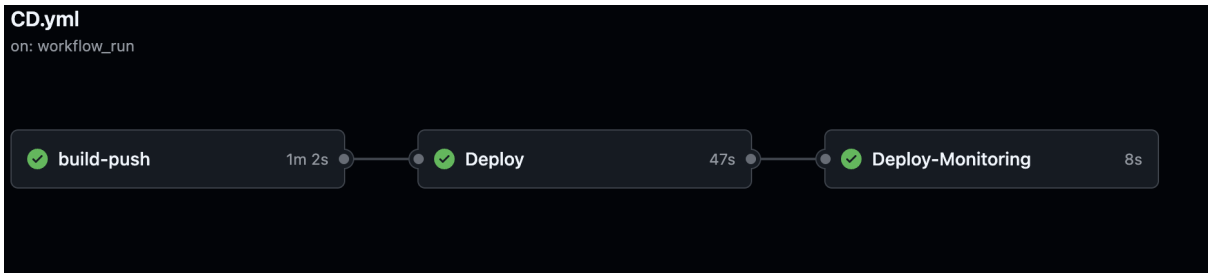

Figure 5: CD pipeline

To ensure high availability of the application, we extended the CD pipeline to deploy the application using **Docker Swarm**, which deploys the application as a swarm with multiple replica nodes while a load balencer distributes traffic across the replicas. this deployment uses rolling updates, which starts a new docker container before closing the previous one to avoid downtime. The application also uses shared **Redis** which caches data across the replicas. The detailed steps of the *Deploy* job are shown in fig. 6:

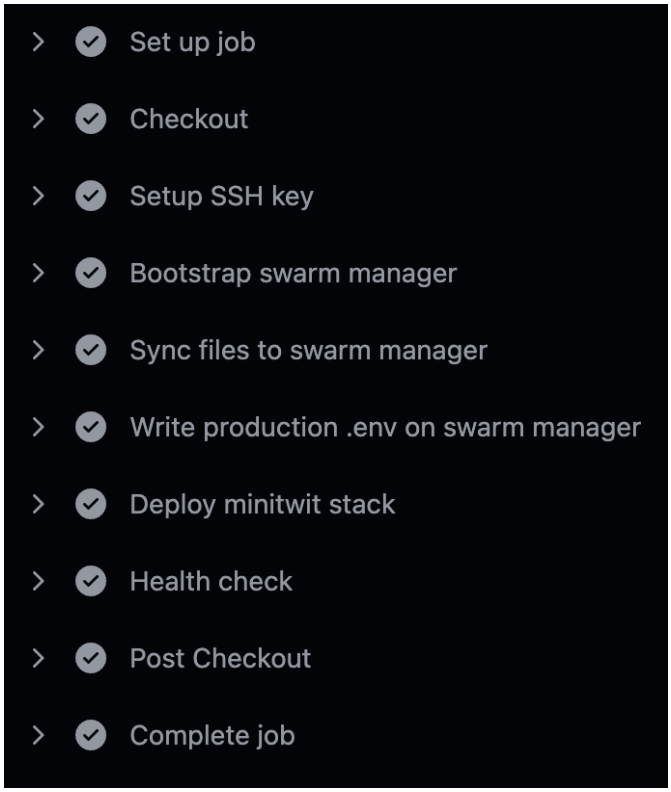

Figure 6: Detailed steps of the *Deploy* job.

2.2 Monitoring

Link to Grafana monitoring dashboard

Metrics are collected through **Prometheus** and visualised in **Grafana**, both running as Docker services on a dedicated monitoring droplet, isolated from the application. Prometheus scrapes application metrics via `'itu-minitwit-app'` defined in the Prometheus configuration and metrics from Node Exporter running on each Swarm node.

We have itterativly upated which data is collected and visualised in the Grafana dashboard based on the needs of the project. At the projects end we have the Grafana dashboard organised into five sections, as shown in fig. 7:

- **Health overview** — application up/down status, average response time, requests / sec, in-flight requests, error rate and current memory usage.
- **HTTP traffic** — request rate by status code, response-time percentiles grouped by p50 p90 and p99. request rate /s grouped by HTTP method, and the response time grouped by api endpoints.
- **Error tracking** error rates req/s grouped by 400/500 http response, errors mesured in req/s per endpoint.
- **Process & runtime** — process CPU usage, process memory usage, thread count, garbage colllection rate.
- **Uptime & Availability** — uptime & availability over time (bar graf), Availability mesured in percentage over the selected time period.

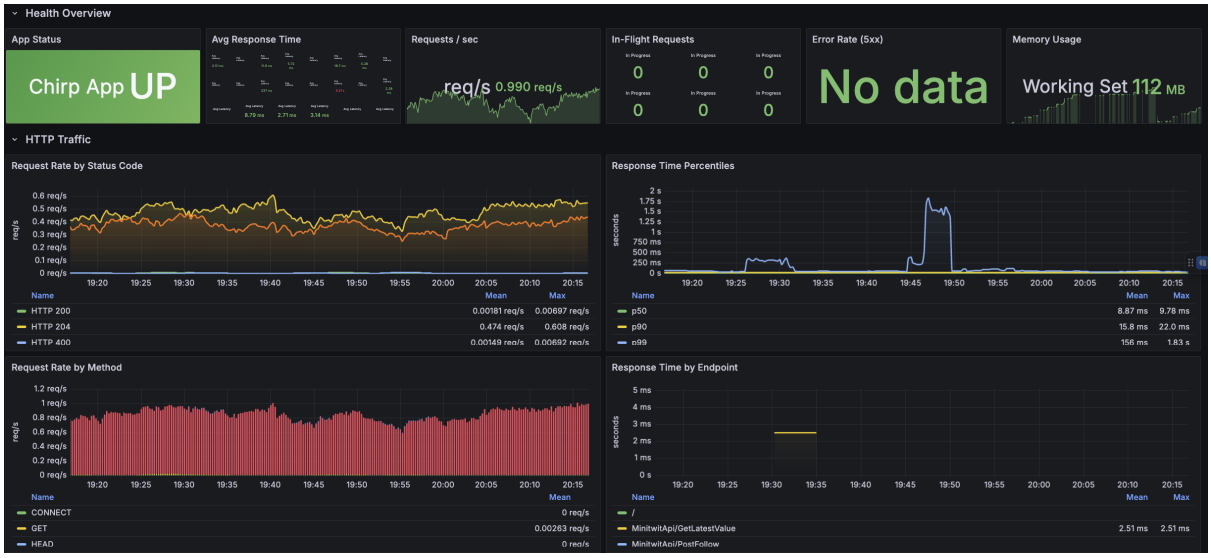

Figure 7: Grafana monitoring dashboard.

2.3 Logging

Link to Grafana logging dashboard

Application logs are aggregated using **Loki** and collected by **Grafana Alloy**. Each line in the logs is labelled with the container name, service name, stream, and image name. Grafana Alloy collects logs from all the replica nodes and forwards them to Loki for aggregation and storage. **Loki** rejects old samples above the "age" of 168hours (7 days) to prevent unbounded log storage growth.

The application logs the following events:

- HTTP requests — method, path, response status code, and elapsed time mesured in ms.
- Unhandled exceptions — method, path, error details.
- Custom logs through `ILogger` throughout the minitwit codebase.

The logs are visualised in a Grafana dashboard with the following panels:

- **Total Log lines & error lines** — Total log lines & errors within the selected time period
- **Log lines & Errors/exceptions & warnings per container** — Log lines & errors grouped by container within the selected time period
- **log volume over time** — Log lines grouped by time (bar graph) within the selected time period
- **log & Error viewer** — Visulisation of all log lines and errors.

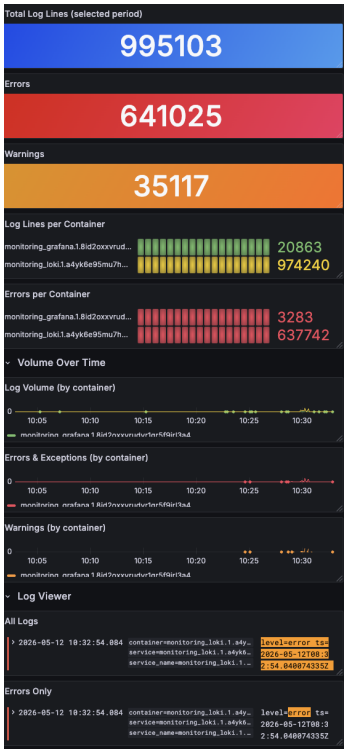

Figure 8: Loki log aggregation in Grafana.

2.4 Security Hardening

We did not have the opportunity to implement hardened docker images as it is a paid feature in Docker Hub, we instead tried to implement parts of the guide OWASP Docker Security Cheat Sheet measures applied from Cheatsheet:

- **Unprivilaged users in Docker containers** — Containers run as an unpriviledged user to prevent privilege escalation attacks.o
- **Prevent in-container privilege escalation** — use –security-opt=no-new-privileges on containers to prevent privilege escalation attacks.
- **Automatic Docker container update** - implementet a WeeklyHostUpdate which automatically updates the docker images on the swarm nodes to ensure that security updates are applied.

System uses applied to harden the system:

- **GitHub Secrets** — database connection strings and API keys are stored as encrypted secrets and injected at deploy time.
- **Firewall** — inbound and outbound traffic on both application droplets and the database is restricted to only the ports and sources required.
- **TLS/HTTPS** — traffic to the application is encrypted using TLS Through the Nginx load balancer using Certbot.
- **Static code analysis** — SonarCloud and trivy are used to identify security vulnerabilities in the codebase/pipelines/docker images.

Static code analysis is integrated in the CI pipeline using **SonarCloud** & **Trivy**. Every build is analysed for bugs, coverage, code duplication and security vulnerabilities. As shown in fig. 9, there are still a number of security vulnerabilities but with only two of five members actively contributing, there was not time to address them before the deadline

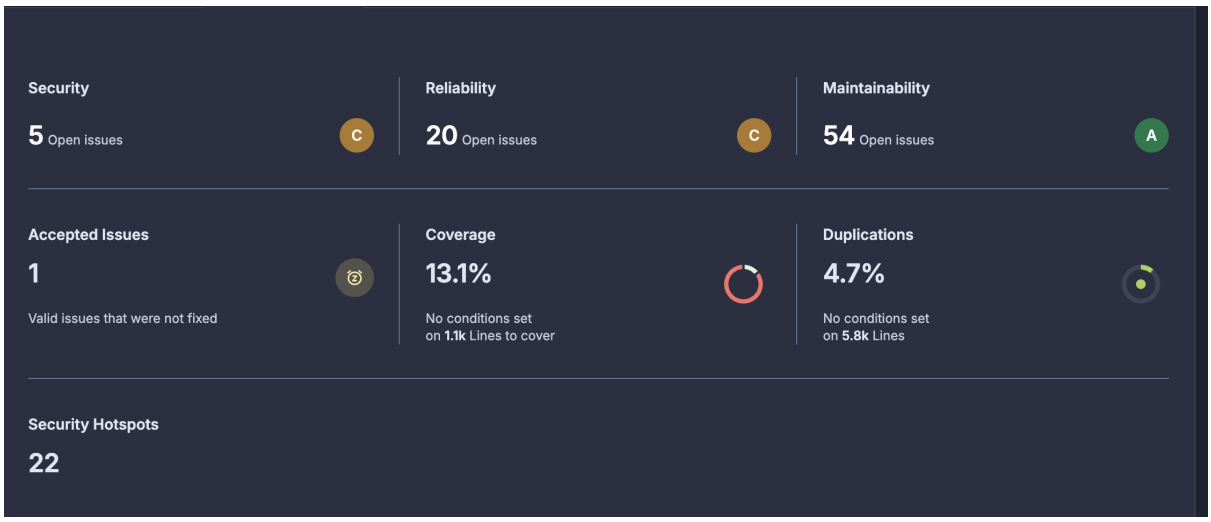

Figure 9: SonarCloud quality gate results.

2.5 Availability and Scaling

Andreas Schiøtt, Sebastian Holt Svendsen

The application is deployed as a **Docker Swarm** stack with **2 replicas** hosted as 2 separate **DigtalOcean droplets** as can be seen in Fig:1. our load balencer distributes traffic across replicas, and automatically reroutes requests away from any node that becomes unavailable/crash. The load balencer performs health checks on the replicas using heartbeats through keepalived, and if a replica becomes unavailable it is automatically removed from the load balencer until it becomes healthy again. Crashed containers are restarted automatically via the on-failure restart policy.

the application uses a **Redis** instance shared between all nodes which caches data between replica nodes, which ensures consistent returned data regardless of which replica handles the request.
Deployments are made using rolling updates, which starts a new docker container before closing the previous one to avoid downtime. this process is repeated for each node with a 10 second delay to ensure that at least one instance is always available for requests during deployments to reduced downtime of the system.

3 Reflection Perspective

Authors: Andreas Schiøtt, Sebastian Holt Svendsen

3.1 Evolution and Refactoring

Our initial implementation of CI/CD pipelines ran concurrently: a push would trigger deployment before its CI run had finished, so failing build could reach production silently. We fixed this by gating CD on a successful CI run, and later added caching to speed up the End-to-End tests, which had been a bottleneck in the process.
In early iterations we did not cull old containers on deployment. The resulting CPU pressure was first visible through our monitoring stack, which then became a crucial diagnostic tool. Monitoring and logging initially ran on the same droplet as the application, a single point of failure that could blind us at the worst moment, so we moved them onto a dedicated droplet, a decision that proved fortunate when that droplet was later compromised (see next paragraph), as it limited the blast radius and kept the application running.

Hacking incident. On 2026-04-21 we discovered that our monitoring droplet had been compromised three days earlier: a brute-force SSH scanning campaign, ongoing since 2026-04-19, had culminated in the `preprod-deploy` key being used for root access from four Azure IPs. The attacker deployed a cryptominer disguised as `/tmp/mysql`, created a fake `dnsmasq` user for persistence, and started a rogue `chirp-postgres` container. Ironically the compromise was discovered not through our monitoring stack but because of it, the droplet was unresponsive and pinned at 100% CPU, leading to a manual investigation. Because root had been obtained directly via the leaked key, the host could not be salvaged. We destroyed and rebuilt it, restricting SSH access via UFW to our own IPs. The full forensic timeline is preserved in `security-incident-ie-BITCOIN-MINE.md`.

Architectual refactor. The system underwent multiple major architectural refactors over the semester, from a locally hosted single-container setup, to a Dockerised application using SQLite, to a DigitalOcean mangaged PostgreSQL, and finally to a Docker Swarm stack with replicated app instances behind a load-balanced floatin IP. Each step was forced by a hard limit of the previous one: SQLite's concurrent write limit, single host failure modes that took the whole system down, and the need for rolling updates without downtime.
Across these refactors the common thread was operational visibility, every significant improvement to the system was the response to a bottleneck or failure discovered either manually or through monitoring.

3.2 Maintenance

We did not implement an alerting system, which left a critical observability gap: the production environment could fail without anyone knowing until we manually checked. The consequences of this were most evident in the security incident described in section 3.1. Implementing an alerting system (at minimum Grafana alerts on uptime, CPU saturation, and authentication failures) would be the highest priority if we continued operating the system. Another instructive incident was self-inflicted. To resolve auth cookies being silently invalidated when load-balanced across replicas, we added `AddDataProtection().PersistKeysToStackExchangeRedis(...)` in `Program.cs`. The fix introduced a new failure mode: `ConnectionMultiplexer.Connect()` is blocking and eager, so the entire application failed to start if Redis was momentarily unavailable. This broke production on first deploy and our E2E test job (which had no Redis instance). We resolved it by adding `abortConnect=false` to the connection string and adding Redis to the E2E workflow. The lesson is that fixes depending on additional infrastructure must be tested against

that infrastructure being absent. Both incidents share the same shape: failures detectable only after the fact, because we built observability for diagnosis but not for prevention. Closing the alerting gap and tightening test coverage of new infrastructure dependencies would be the highest maintenance priorities.

3.3 DevOps Style of Work

Compared to previous software projects, our development process was substantially more iterative and incremental, with continuous integration and delivery as the default rather than an afterthought. The most consequential shift was treating observability as a first-class feature. We built monitoring and logging early, and tried using them a our primary feedback loop. This habit has carried over into our other projects and courses.
When it comes to working DevOps style, a DevOps team is most effective when ownership, on call responsibility, and review duties are distributed across all members. Ours was not.
Of our five-person group, two of us carried the substantive work for the full semester. The others didn't commit much code, take branches, nor return to tasks taken on. This was not a deliberate decision, but rather an unfortunate outcome of the group dynamics. Some of this we own: in the first weeks we moved quickly and likely raised the bar for participation, probably higher than what was inclusivem, without doin the team-formation work (shared expectations, paired tasks, defined responsibilities, etc.) that would have made on-ramping possible.
The operational cost was real, a bus-factor of two on every piece of infrastructure, the missing alerting in section 3.2

Much more iterative and incremental development process, with a focus on continuous integration and delivery. with a larger focus on introspection and adaption to feedback from monitoring and logging, which allowed us to detect and fix issues in production.

This way of working has a made a large impact on our development process and has changed our approch to software development in our other projects and courses.

4 Use of Generative AI

Authors: Andreas Schiøtt, Sebastian Holt Svendsen

4.1 Tools Used

- Claude / Claude Code(CLI) For Brainstorming, debugging, Understanding course material, code generation, documentation, code reviews/Refactoring, process improvement suggestions.
- ChatGPT (OpenAI) For Brainstorming, debugging, Understanding course material, code generation, documentation, code reviews/Refactoring, process improvement suggestions.
- GitHub Copilot for code suggestions and autocompletion

4.2 Impact on Work Process

4.2.1 Supported

It helped us understand course material and concepts better by providing explanations and examples / deepening the explanations from the lectures/course material. able to get less stuck on problems, and get suggestions for how to solve them, which sped up development. helped with code generation for ci/cd pipelines, dockerfiles, and other boilerplate code.

4.2.2 Hindered

As mentioned earlier, we have been 2 people working on the project, this resulted in us being behind on the project, which meant that the use of generative AI could sometimes could become a crutch to catch up. To get value of the use of generative AI had to give alot of context to get usefull suggestions / answers /avoid hallucinations which made output useless. This has been helped by the use of CLI based tools like Claude Code/ Codex which could take the context from the codebase.

References